Trabalho prático número 02
Trajetória de um robô

Um robô autônomo desloca-se em um plano cartesiano a partir da posição inicial (x, y). Seus movimentos são representados por comandos unitários nas direções Norte, Sul, Leste e Oeste, armazenados inicialmente em uma lista. O objetivo do projeto é implementar um sistema completo de simulação, análise, otimização e comparação de trajetórias utilizando diferentes estruturas de dados. Os comandos possíveis são:

`'N'`	Deslocamento da posição `(x, y)` para a posição `(x, y + 1)`, ou seja, para **cima**.
`'S'`	Deslocamento da posição `(x, y)` para a posição `(x, y - 1)`, ou seja, para **baixo**.
`'L'`	Deslocamento da posição `(x, y)` para a posição `(x + 1, y)`, ou seja, para **direita**.
`'O'`	Deslocamento da posição `(x, y)` para a posição `(x - 1, y)`, ou seja, para **esquerda**.

Ademais, cada comando deverá ser representado por:

```
typedef struct Comando {
    char Direcao; // Que pode ser ‘N’, ‘S’, ‘L’ ou ‘O’
    struct Comando * Proximo;
} Comando;
```

Para armazenar cada posição por onde o robô passará será usada a seguinte estrutura:

```
typedef struct Posicao {
    int X; // Para armazenar a abscissa de um ponto
    int Y; // Para armazenar a ordenada de um ponto
    struct Posicao * Proximo;
} Posicao;
```

Para armazenar um caminho entre a posição inicial e a posição final, deve ser usado o tipo abstrato de dados lista, conforme detalhado a seguir:

```
typedef struct Caminho {
    Posicao * Inicio; // Ponteiro para a primeira posição ocupada pelo robô
    Posicao * Fim; // Ponteiro para a última posição ocupada pelo robô
    Posicao * Historico; // Ponteiro para o histórico de posições ocupadas pelo robô
    Comando * Instrucoes; // Ponteiro para a primeira das instruções a serem executadas
    int N; // Quantidade de instruções executadas no trajeto
} Caminho;
```

Objetivo geral

Implementar, em linguagem C, um sistema capaz de armazenar, processar, otimizar e analisar trajetórias de um robô em um plano cartesiano, explorando o uso do tipo abstrato de dados lista, em implementação usando ponteiros (e respectivos encadeamentos), conforme tratado em sala de aula entre os dias **04/05/2026** e **15/05/2026**.

Requisitos obrigatórios

O sistema deverá:

1. Inicializar um caminho a partir da sequência de comandos em uma string (**`const char *Sequencia`**) e das coordenadas, ou seja, da abscissa (**`xInicial`**) e da ordenada (**`yInicial`**) da posição inicial do robô, conforme destacado no protótipo a seguir:

   ```
   Caminho * InicializarCaminho (const char *Sequencia, int xInicial, int yInicial);
   ```

   Observação:

   Todos os campos da *struct Caminho* devem ser inicializados e/ou calculados.

2. Liberar a memória alocada para o caminho C, bem como de suas estruturas internas (isto é, os ponteiros Inicio, Fim e Instruções), a partir da referência C, conforme especificado no protótipo a seguir:

   ```
   void DestruirCaminho(Caminho * C);
   ```

3. Calcular a posição final do robô após a execução dos comandos presentes em uma lista de instruções, ou seja, no campo correspondente em uma variável do tipo caminho (**`struct Caminho`** ou **`Caminho`**), conforme especificado no protótipo a seguir:

   ```
   Posicao * DeterminarFim(Caminho * C);
   ```

4. Determinar o encadeamento com todas as posições visitadas pelo robô no deslocamento que efetuar. Nesse caso, o valor de retorno será o endereço de memória da posição inicial do robô e os encadeamentos devem refletir a ordem em que as posições foram visitadas.

   ```
   Posicao * HistoricoPosicoes(Caminho * C);
   ```

5. Determinar a distância total percorrida pelo robô, definida como a soma dos comprimentos dos deslocamentos elementares realizados durante a execução da sequência de comandos, considerando que cada movimento corresponde a uma translação unitária no plano cartesiano. Para cada caminho passado como parâmetro, essa função deve ter o seguinte protótipo:

   ```
   int CalcularDistanciaTotal(Caminho * C);
   ```

6. Determinar a distância euclidiana entre as posições inicial (x_1, y_1) e final (x_2, y_2) do robô. É importante destacar que essa distância corresponde à distância, em linha reta, entre os pontos (x_1, y_1) e (x_2, y_2), podendo ser calculada a partir da fórmula:

   $$d_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

   Além disso, a distância geométrica deve seguir o protótipo:

   ```
   double CalcularDistanciaGeometrica(Caminho * C);
   ```

7. A *distância de Manhattan* é uma métrica utilizada para medir a distância entre dois pontos em um plano cartesiano quando os deslocamentos são restritos às direções horizontais e verticais, e seu nome deriva do traçado urbano em forma de grade do bairro de Manhattan, no qual o deslocamento entre dois pontos ocorre ao longo de ruas e avenidas, e não em linha reta.

Formalmente, dados dois pontos (x_1, y_1) e (x_2, y_2), a distância de Manhattan é definida como a soma dos valores absolutos das diferenças entre suas coordenadas, ou seja:

$$d_M = |x_1 - x_2| + |y_1 - y_2|$$

Essa métrica representa a menor quantidade de movimentos unitários necessária quando são permitidos apenas deslocamentos horizontais e verticais.

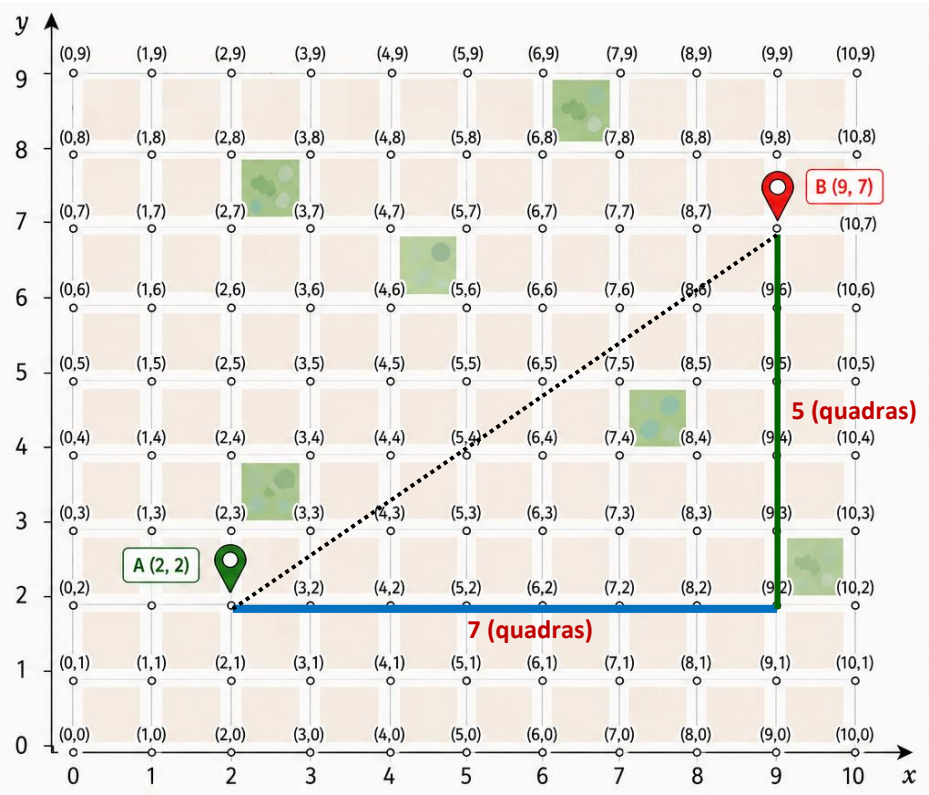

Figura destacando a distância de Manhattan (soma dos catetos) e a distância geométrica (hipotenusa).

No contexto do problema do robô, a distância de Manhattan entre a posição inicial e a posição final indica o caminho ótimo em termos de número mínimo de comandos, desconsiderando eventuais movimentos redundantes realizados durante a execução. Por exemplo, se o robô começa em (2, 2) e termina em (9, 7), a distância de Manhattan é dada por $|9 - 2| + |7 - 2| = 7 + 5 = 12$, o que significa que seria possível atingir a posição final com apenas doze movimentos, ainda que a sequência original contenha mais comandos.

Ao implementar a função para calcular a distância de Manhattan, deve-se usar o seguinte protótipo, cujo cálculo deve levar em consideração os atributos da variável (ponteiro) C:

```
int CalcularDistanciaManhattan(Caminho * C);
```

8. Determinar o número de instruções executadas no caminho do robô, isto é, o número de ações (deslocamentos unitários) para sair do ponto de início e chegar no ponto final. Para este requisito, deve-se usar implementar função cujo protótipo é:

```
int ContarInstrucoes(Caminho * C);
```

Atenção

1. Comandos (ou instruções) inválidas devem ser tratadas como erro, conforme implementação de cada aluno e dentro da função `InicializarCaminho`, implicando no retorno NULL como referência para o caminho esperado;

2. Na implementação das funcionalidades propostas acima, **não é permitido** o uso de vetores para armazenar o histórico de posições ou os comandos a serem desenvolvidos pelo robô.

3. Ocorrendo problemas ou inconsistências na execução de cada função:

   - funções que retornam ponteiros devem retornar NULL;
   - funções do tipo int devem retornar -1;
   - funções do tipo double devem retornar -1.

4. O código-fonte deve ser desenvolvido utilizando o padrão da linguagem C, a ser compilado e executado em computadores com o sistema operacional Windows, típica dos equipamentos disponibilizados nos laboratórios da UCB;

5. Este trabalho prático é individual, vale 1,0 ponto e, caso seja entregue com atraso, será aplicada a penalização de 10% por dia de atraso, passando a valer $\mathbf{1,0 \cdot 0,9^n}$, em que n é o número de dia de atraso;

6. Neste trabalho prático, você deverá utilizar, obrigatoriamente, o seguinte arquivo de cabeçalho, que deve ser incluído no seu projeto a partir **#include "caminho.h"**. É importante ressaltar que esse arquivo, em hipótese alguma, pode ser modificado.

```
#ifndef CAMINHO_H
#define CAMINHO_H

typedef struct Comando {
    char Direcao; // Que pode ser 'N', 'S', 'L' ou 'O'
    struct Comando * Proximo;
} Comando;

typedef struct Posicao {
    int X; // Para armazenar a abscissa de um ponto
    int Y; // Para armazenar a ordenada de um ponto
    struct Posicao * Proximo;
} Posicao;

typedef struct Caminho {
    Posicao * Inicio; // Ponteiro para a primeira posição ocupada pelo robô
```

```
    Posicao * Fim; // Ponteiro para a última posição ocupada pelo robô
    Posicao * Historico; // Ponteiro para o histórico de posições ocupadas pelo robô
    Comando * Instrucoes; // Ponteiro para a 1ª das instruções a serem executadas
    int N; // Quantidade de instruções executadas no trajeto
} Caminho;

Caminho * InicializarCaminho(const char *Sequencia, int xInicial, int yInicial);
void DestruirCaminho(Caminho * C);
Posicao * DeterminarFim(Caminho * C);
Posicao * HistoricoPosicoes(Caminho * C);
int CalcularDistanciaTotal(Caminho * C);
double CalcularDistanciaGeometrica(Caminho * C);
int CalcularDistanciaManhattan(Caminho * C);
int ContarInstrucoes(Caminho * C);

#endif
```

Importante:

Elementos adicionais como funções, variáveis ou estruturas podem ser usados de forma auxiliar, desde que restritos ao arquivo que você entregar.

7. A seguir, é é mostrado um exemplo de utilização dessas funções (main):

```
#include <stdio.h>
#include <stdlib.h>
#include "caminho.h"

void ExibirHistorico(Posicao *P) {
    while (P != NULL) {
        printf("(%d, %d)", P->X, P->Y);
        if (P->Proximo != NULL) printf(" -> ");
        P = P->Proximo;
    }
    printf("\n");
}

int main() {
    Caminho * C;
    Posicao * Fim;
    Posicao * Historico;

    /*
        Sequência escolhida: LLLLLLLNNNNN
        Posição inicial: (2, 2)
        Movimentos:
        7 comandos Leste  -> x aumenta de 2 para 9
        5 comandos Norte  -> y aumenta de 2 para 7
        Posição final esperada: (9, 7)
    */
```

```
    C = InicializarCaminho("LLLLLLLNNNNN", 2, 2);

    if (C == NULL) {
        printf("Erro ao inicializar o caminho.\n");
        return 1;
    }

    // Teste da função DeterminarFim(C), cujo resultado esperado é Fim = (9, 7)

    Fim = DeterminarFim(C);
    if (Fim != NULL) printf("Posicao final: (%d, %d)\n", Fim->X, Fim->Y);

    /* Teste da função HistoricoPosicoes(C), cujo resultado esperado é (2, 2) -> (3,
    2) -> (4, 2) -> (5, 2) -> (6, 2) -> (7, 2) -> (8, 2) -> (9, 2) -> (9, 3) -> (9, 4)
    -> (9, 5) -> (9, 6) -> (9, 7) */

    Historico = HistoricoPosicoes(C);
    printf("Historico de posicoes:\n");
    ExibirHistorico(Historico);

    /* Teste da função CalcularDistanciaTotal(C): Como existem 12 comandos unitários,
    o resultado esperado é 12. */

    printf("Distancia total percorrida: %d\n", CalcularDistanciaTotal(C));

    /* Teste da função CalcularDistanciaGeometrica(C): no caso, a distância euclidiana
    entre (2,2) e (9,7) é sqrt((9 - 2)^2 + (7 - 2)^2) = 8.60 */

    printf("Distancia geometrica: %.2f\n", CalcularDistanciaGeometrica(C));

    /* Teste da função CalcularDistanciaManhattan(C): no caso, o valor esperado é |9 -
    2| + |7 - 2| = 7 + 5 = 12 */

    printf("Distancia de Manhattan: %d\n", CalcularDistanciaManhattan(C));

    /* Teste da função ContarInstrucoes(C): A sequência LLLLLLLNNNNN possui 12
    comandos. */

    printf("Quantidade de instrucoes: %d\n", ContarInstrucoes(C));

    /* Teste da função DestruirCaminho(C): o resultado esperado envolve a liberação de
    toda a memória alocada para o caminho, posições e comandos*/

    DestruirCaminho(C);

    return 0;
}
```

8. Você deverá enviar somente um arquivo não compactado do tipo C, cujo nome deve ser formado pelos dígitos do seu número de matrícula seguido de “.c”. Por exemplo, se seu número de matrícula

é **UC26209999**, o arquivo a ser enviado deve ter **26209999.c** como nome. O mesmo será compilado usando a seguinte instrução, considerando arquivo main.c presente nesta proposta de projeto:

gcc 26209999.c main.c -o robo.exe

Atente-se bem a essas instruções pois a correção será feita de forma automática, cujas funções serão usadas de forma similar à apresentada na função main detalhada acima.

9. A correção deste trabalho prático levará em consideração os seguintes aspectos:

- Funcionamento correto de cada uma das funções especificadas nesta proposta;
- Uso adequado de alocação dinâmica de memória;
- Manipulação correta de ponteiros e encadeamentos;
- Compatibilidade com o arquivo de cabeçalho caminho.h, sem modificações em sua estrutura;
- Utilização apropriada de listas encadeadas e conceitos associados ao TAD lista;
- Ausência de vazamentos de memória (*memory leaks*) e liberações indevidas de memória;
- Tratamento adequado de erros e comandos inválidos;
- Organização, legibilidade e indentação do código-fonte;
- Compatibilidade da implementação com o compilador GCC no ambiente especificado;
- Respeito às restrições e exigências descritas nesta proposta de trabalho.

Além dos critérios anteriores, poderão ser considerados aspectos relacionados à eficiência da solução implementada e à clareza na organização lógica do código-fonte.

10. Trabalhos semelhantes podem receber nota igual a 0. Para avaliar o grau de semelhança entre os arquivos enviados, podem ser usadas ferramentas disponíveis de mercado e/ou scripts próprios desenvolvidos para essa finalidade.